Clase 05: Actividad — Efecto Caché

IIC3533 Computación de Alto Rendimiento

Ignacio Labarca – ignacio.labarca@uc.cl

Departamento de Ciencia de la Computación
Escuela de Ingeniería
Pontificia Universidad Católica de Chile

Jueves 19 de Marzo de 2026

Requisitos

- **Sistema operativo:** Linux o Mac OS X
- **Windows:** Windows Subsystem for Linux (WSL)
  https://learn.microsoft.com/windows/wsl/install

Requisitos

- Compilador para C++ con soporte para C++17
  - `gcc` $\geq$ 7
    - Ubuntu: `apt install g++`
    - Mac OS X: `brew install gcc`
  - `clang` $\geq$ 5
    - Ubuntu: `apt install clang`
    - Mac OS X: incluido en Xcode Command Line Tools

Opcional

- CMake $\geq$ 3.15 https://cmake.org/download
  - Ubuntu: `apt install cmake`
  - Mac OS X: `brew install cmake`

Requisitos

- Python 3 con las librerías `numpy`, `matplotlib`, `seaborn`
- Recomendado: usar un entorno virtual con Miniconda3
  https://docs.anaconda.com/miniconda
  - `conda create -n iic3533 numpy matplotlib seaborn`
  - `conda activate iic3533`

Compilación

- Con gcc:

```
g++ -std=c++17 -march=native -fno-tree-vectorize cache.cpp -o cache
g++ -std=c++17 -march=native latency.cpp -o latency
```

- Con clang:

```
clang++ -std=c++17 -march=native -fno-vectorize cache.cpp -o cache
clang++ -std=c++17 -march=native latency.cpp -o latency
```

- Alternativamente, con CMake:

```
cmake -S . -B build
cmake --build build
```

Ejecución

Usamos `hwloc-bind` para fijar el proceso a un *core*.

Reduce ruido en las mediciones.

En Apple Silicon: revisar topología del hardware con `hwloc-ls`.

- Con `gcc` o `clang`:

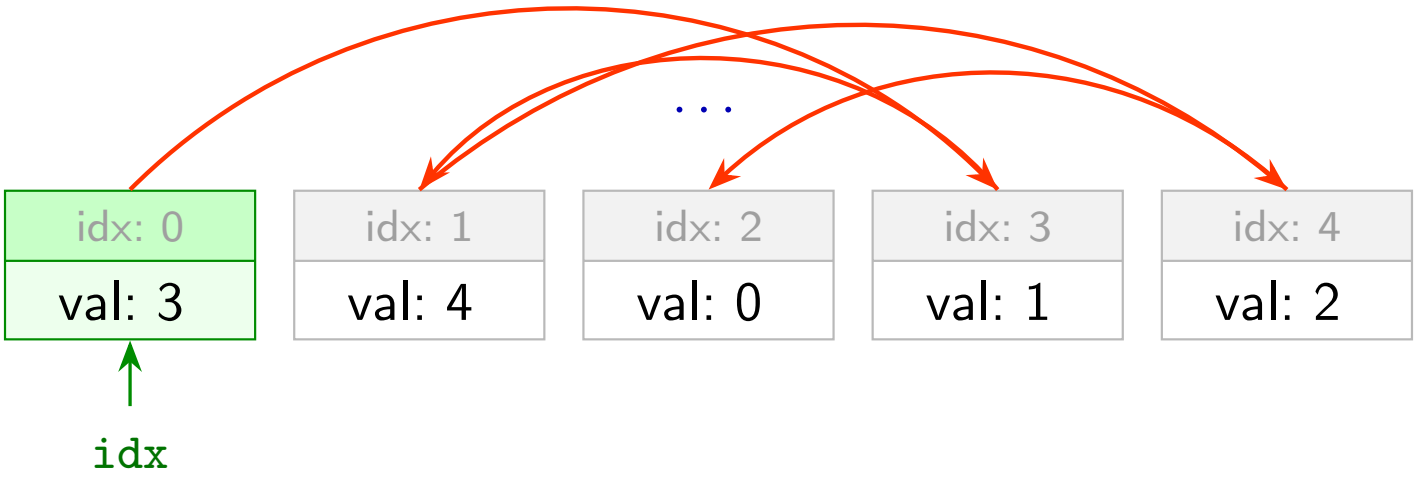
```
hwloc-bind core:0 -- ./cache > cache_data.csv
hwloc-bind core:0 -- ./latency > latency.csv
```

- Con CMake:

```
cmake --build build --target cache_data latency_data
```

Genera `build/cache_data.csv` y `build/latency.csv` automáticamente.

Pointer chasing: recorrido impredecible



Dependencia de datos: el paso $n+1$ requiere el valor leído en el paso n

Escalera de latencias

Pointer chasing: cada posición contiene el índice de la siguiente.

```
// arr es una permutacion aleatoria de indices
for (long long s = 0; s < STEPS; s++)
    idx = arr[idx];
```

- El prefetcher no puede predecir el siguiente acceso.
- Al variar el tamaño de `arr` se revela la latencia de cada nivel.

Efecto caché

Acceso con stride fijo: recorre el arreglo saltando de `stride` en `stride`.

```
1 for (size_t sweep = 0; sweep < num_sweeps; ++sweep)
2     for (size_t i = 0; i < N; i += stride)
3         arr[i] += 1;
```

- Al variar `N`: se revelan los límites de cada nivel de caché.
- Al variar `stride`: se revela el tamaño de la línea de caché y la asociatividad.

Graficar resultados

- Directamente:

```
python3 plot.py cache_data.csv latency.csv
```

  Genera `result/plot_a.pdf`, `plot_b.pdf` y `plot_c.pdf`.

- Con CMake:

```
cmake --build build --target plot
```

  Genera los PDFs en `build/result/`.

Actividad

En grupos de hasta 4 personas, contestar las siguientes preguntas y entregar en un PDF por Canvas:

- Indique las especificaciones de hardware de su computador: tamaño de caché, línea de caché y asociatividad.
- Adjunten los gráficos generados y comenten sus resultados. ¿Cómo se relacionan con los datos del punto anterior?
- ¿Qué pasa si N/stride es mayor a la asociatividad? Justifique en base al concepto de *cache miss* por conflicto.